Checking AI-Generated Proofs:
The Human Friendly Form

Siddharth Iyer

October 2026

Abstract

Suppose an AI system has produced a proof of a theorem T. This note proposes a way for a human to convince themselves that the proof is correct by reading it, without relying on a proof assistant. The AI is asked to write the proof in a prescribed format, the *Human Friendly Form* (HFF): a plain-English overview, then, after a distinguished mark, a proof written entirely as definitions, lemmas and remarks, in which every lemma is proved directly below its statement in fewer than a fixed number of visible characters, and finally a list of references. Alongside the document, the AI produces a *map*: an interactive two-dimensional rendering of the dependency graph of the definitions and lemmas, which the reader explores as one explores the map of a strategy game. Checking the proof then reduces to checking a list of short, independent steps in any order, and the map records which steps have been checked.

1 Introduction

Suppose that an AI system has produced a proof of a theorem T. When is a human entitled to be happy with it? In this note we take the answer to be: when they have convinced themselves that the proof is correct, by checking it as a careful referee checks a paper. We do not assume that the proof has been formalised in a proof assistant, and we set aside the harder question of what it would mean to understand the proof, as well as the broader question, taken up by Tao [5], of what the goals and values of mathematical research become once AI tools can do research-level mathematics. The question of correctness is a pressing one. The recent Leiden Declaration on Artificial Intelligence and Mathematics [1] warns that automated tools can produce arguments that look plausible but are unreliable or even wrong, and that are hard to tell apart from correct proofs; it also holds that responsibility for the correctness of a result stays with its human authors. The aim of this note is to make that responsibility practical to discharge by hand.

Checking an AI-generated proof in its raw form can be hard for reasons that have little to do with the mathematics. The argument may be long and unbroken, auxiliary claims may be proved inline and used much later, routine and delicate steps may be interleaved without signposting, and the dependencies between the parts may be left implicit. The referee then has to hold large parts of the argument in mind at once, and an error can hide in the middle of a long paragraph.

On the other hand, rewriting a proof into a different shape costs the AI almost nothing, while the reader's attention is scarce. We therefore ask the AI to write its proof in a form designed for checking, the *Human Friendly Form* (HFF). It has three ingredients:

- an **overview** in plain English, so that the reader knows what is proved and by what strategy before checking any detail;
- a **map** of the dependency graph of the proof, through which the reader navigates and on which they record their progress;
- a proof written entirely in **small blocks**, with every lemma proved in fewer than L visible characters, so that each block can be checked in a single sitting.

The last ingredient rests on a simple local-to-global principle (Proposition 5.2): if every block has been checked, taking the statements it cites as given, then T has been proved. Checking the proof becomes a list of small, independent tasks.[1]

2 The Human Friendly Form

Definition 2.1 (Blocks)**.** A *block* is one of the following.

(a) A *definition*: a header and a body introducing a notion or a piece of notation.

(b) A *lemma*: a header, a statement, and a proof placed immediately below the statement. The main theorem T is a distinguished lemma, typeset as "Theorem".

(c) A *remark*: a header and free text.

Blocks are numbered consecutively in a single sequence, whatever their kind, so that each number names exactly one block: a document might begin with Definition 1, Lemma 2, Remark 3. The *header* of a block is its number together with a short descriptive name, for instance "Lemma 5 (Prime factor $\equiv 3 \bmod 4$)".

Definition 2.2 (Citations and the dependency graph)**.** A block v *cites* a definition or lemma u if the statement or the proof of v uses u; citations must be explicit, by number. The *dependency graph* G of a document is the directed graph whose vertices are its definitions and lemmas, with an edge $u \to v$ whenever v cites u. Remarks are not vertices of G.

Definition 2.3 (Human Friendly Form)**.** A document is in *Human Friendly Form* for T, with budget L, if it satisfies the following conditions.

(H1) Overview. It opens with *Part I*, at most a few pages of English prose, which states T in words and describes the methods used to prove it: the overall strategy, the techniques involved, and which steps are routine and which are delicate.

(H2) Separator. Part I is followed by a line containing only the mark ✠. Everything after the mark, up to the references, is *Part II*, the proof.

(H3) The map. The document is accompanied by a separate file, the *map*, which renders G interactively (Section 3). A static drawing of G may also be placed directly after the mark, but it is optional.

[1] Throughout, T denotes the main theorem and L the length budget for proofs, counted in visible characters of the compiled document (Section 4). A reasonable default is $L = 300$, about four or five typeset lines.

(H4) Block discipline. In Part II, every piece of text belongs to a definition, a lemma or a remark. There is no free-standing prose; the only other thing allowed is the optional drawing of G.

(H5) Short proofs. The proof of each lemma appears directly below its statement and has length less than L, where length is the number of visible characters in the compiled document (Section 4).

(H6) Well-founded order. Each block cites only blocks that appear before it. In particular G is acyclic, and the order of the document is a topological order of G.

(H7) Single goal. T is the last lemma, and every vertex of G has a directed path to T.

(H8) Inert remarks. Remarks carry no logical weight: deleting every remark leaves a complete proof of T. In particular, no block cites a remark.

(H9) References. The document ends with a list of references, as any paper does, whenever it draws on the literature. Part I and the remarks may cite it freely, and every imported lemma (Remark 2.5) must.

Figure 1 shows the shape of such a document.

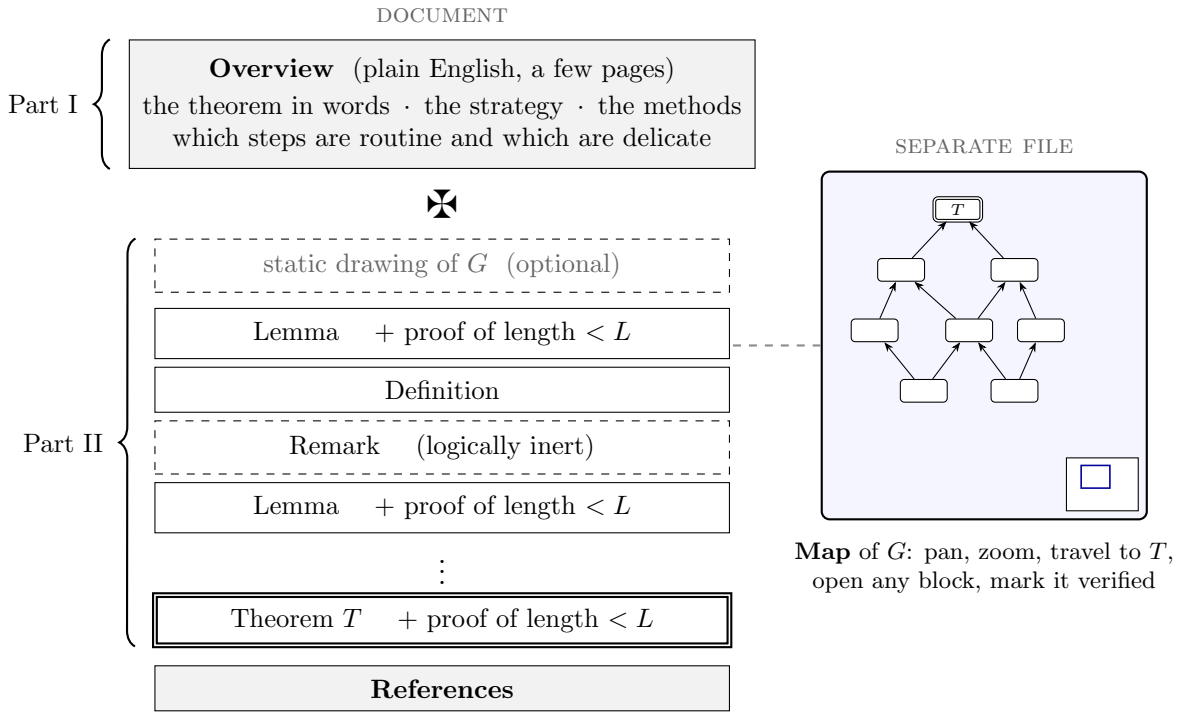


Figure 1: The anatomy of an HFF document and its map. In Part II, definitions, lemmas and remarks may appear in any order in which each block cites only earlier blocks **(H6)**; the order shown is one possibility, and the main theorem always comes last **(H7)**.

Remark 2.4 (What the conditions are for). Conditions **(H4)** and **(H5)** make every step small. Conditions **(H6)** and **(H7)** make G acyclic with T as its only sink, which is exactly what is

needed for checking every block to suffice (Proposition 5.2); without **(H6)**, a cycle in G would be a circular argument hiding in plain sight. Condition **(H8)** lets the reader skip every remark while checking, since remarks carry only motivation and heuristics. Condition **(H9)** is the convention of every paper, stated only because **(H4)** might otherwise seem to forbid a bibliography.

Remark 2.5 (Imported results). A proof of T will usually rely on theorems from the literature, which cannot be reproved within the budget. We therefore allow an *imported lemma*: a statement with a precise citation of the reference list, including the theorem number, in place of a proof. Imported lemmas are leaves of G and are drawn distinctly on the map. They mark the trust boundary of the document, and the reader must check that each imported statement agrees *exactly* with its source, hypotheses included; this is a common place for errors to hide.

Remark 2.6 (No hidden content in definitions). Definitions have no proofs, which makes them a natural hiding place for unproved claims, as in "let f be the unique function such that…". Any existence, uniqueness or well-definedness claim implicit in a definition must therefore be a separate lemma, which the definition cites and which, by **(H6)**, appears before it.

Remark 2.7 (The map is checked against the text). Since G is determined by the citations, the map need not be taken on trust: its data can be generated from the source of the document by a script, or, if the AI writes it, compared with the source by a script. This is why Definition 2.2 insists that citations be explicit: "by the previous lemma" should read "by Lemma 4".

3 The map

A drawing of G on a printed page works for a dozen vertices. The dependency graph of a serious proof may have hundreds, and is then better explored than looked at. The model is the map of a real-time strategy game such as *Age of Empires* [2]: a large two-dimensional world of which the screen shows one window at a time, with a minimap in the corner showing the whole world and where the window currently is. The reader moves around with the mouse, the trackpad, a touch screen or the keyboard.

Definition 3.1 (Map)**.** A *map* of an HFF document is an interactive rendering of its dependency graph G with the following features.

(M1) Layout. Each vertex v is placed in a horizontal layer according to its *depth*, the length of the longest directed path from v to T, with T alone at the top. Every edge points upward, so moving up the map means moving towards the theorem.

(M2) Navigation. The reader pans by dragging and zooms with the scroll wheel or a pinch. A minimap shows the whole graph and the current window. A single key travels to T, and a search box travels to any block by number or by words in its header.

(M3) Semantic zoom. Zoomed out, each vertex shows only its number; at middle range, its header; zoomed in, its full statement.

(M4) Opening a block. Selecting a vertex opens its block beside the map: statement, proof, the length of the proof, and the blocks it uses and that use it, each a link that moves the window there. The paths from the vertex up to T are highlighted.

(M5) Verification status. Each vertex is marked *unverified* or *verified* as the reader goes. Every unverified vertex lies under a dense fog, as unexplored land does in a strategy

game. When a vertex is verified the dense fog lifts, but a *light* fog remains over it for as long as some vertex it depends on, directly or indirectly, is unverified: its proof has then been checked only relative to statements that have not. There is no fog anywhere else on the map, and it cannot be switched off.

By the argument of Proposition 5.2 below, applied to a vertex together with the vertices it depends on, a vertex with no fog over it is proved, relative to the imported lemmas. In particular the map is clear of fog exactly when every vertex has been verified, and then T is proved.

Remark 3.2 (Data, not drawing). The map is best split into a *data file* listing every block, written by the AI and checked against the document (Remark 2.7), and a *viewer* that renders any such file, written once and shared by every HFF document. One entry of the data file might read as follows, in JSON.

```
{ "id": "L5", "kind": "lemma", "number": 5,
  "header": "Prime factor $\\equiv 3 \\bmod 4$",
  "statement": "If $n > 1$ and $n \\equiv 3 \\pmod{4}$, then ...",
  "proof": "By Lemma 2, $n = p_1 \\cdots p_r$ with ...",
  "length": 151, "cites": ["D1", "L2", "L3"] }
```

The references are listed separately. Remarks are left off the map: they carry no logical weight **(H8)**, so they play no part in checking, and they remain in the document for anyone who wants them. The depths, the layout and the highlighted paths are computed by the viewer.

Remark 3.3 (Static drawings and stable layouts). For a small graph, say at most fifteen vertices, a static drawing after the mark is still worthwhile, since it can be printed. For any graph, the layout should be stable: when the document changes, for instance when a lemma is refined (Section 5.3), new vertices should appear near the vertex they replace and the rest of the map should not move, so that the reader can still find their way around.

4 Measuring length

Condition **(H5)** needs a notion of the length of a proof. The purpose of the budget is that each proof can be held in mind all at once and checked in a single sitting, so length should measure what the reader actually reads.

The source of a proof is the wrong thing to measure, because its length depends on the typesetting language in which the HFF document is written and compiled (LaTeX, Markdown, Typst, HTML, and so on) and on the macros its author happens to use. The formula $\sum_{n \le x} \Lambda(n) = x + O(\sqrt{x})$ takes 43 characters of LaTeX source but shows the reader only 16 symbols, and `\mathbb{R}` is ten characters of source for a single symbol on the page. The same proof, written in two languages or with two sets of macros, would have two different source lengths. We therefore measure the compiled document, which is what the reader sees.

Definition 4.1 (Length of a proof)**.** Compile the HFF document. The *length* $|P|$ of a proof P is the number of visible characters in its compiled form: every letter, digit, punctuation mark and mathematical symbol that appears on the page counts as one, whether it occurs in running text or in a formula, inline or displayed. Spaces do not count, and neither do the label "Proof" and the end-of-proof mark. A proof is *within budget* if $|P| < L$.

Remark 4.2 (No exemption for formulas). Formulas are counted exactly like words. An exemption for formulas, however natural it may seem, would let a proof move its content into displayed

equations, which the reader must check just as carefully as any sentence. A long calculation must therefore be split into lemmas, each a single identity or estimate. Long calculations are exactly where errors hide, and splitting them lets each piece be checked on its own.

Remark 4.3 (Measuring in practice). Length does not depend on the typesetting language: two sources that compile to the same page have the same length. It can be measured by a script, for instance by extracting the text of each proof from the compiled PDF and counting its non-space characters. Extraction treats a few constructions in its own way (the three dots of $\cdots$ count as three characters, while the bar of a fraction or of a square root is not a character at all), but such differences are small and are the same for every proof.

Remark 4.4 (Calibration and loopholes). The value $L = 300$ is a starting point, to be calibrated by experience; since spaces are not counted, a proof of 300 visible characters fills about four or five typeset lines. Fields in which proofs are formula-heavy, such as analysis, may want a larger budget. Any fixed rule can be met in unhelpful ways, for instance by dense notation or by packing work into long lemma statements. A soft budget on statements and definitions (say $2L$) helps, and refinement (Section 5.3) lets the reader push back on any block that is short but not convincing.

5 Checking an HFF document

5.1 Checking is local

Definition 5.1 (Local check)**.** A reader has *checked*, or *verified*, a lemma v when they have convinced themselves that the proof of v establishes its statement, taking the statements of the blocks that v cites as given. A definition is checked when the reader has confirmed that it is well formed, any claims it relies on being cited lemmas (Remark 2.6). An imported lemma is checked once its statement has been compared with the source.

Proposition 5.2 (Local to global)**.** *If every vertex of G is checked, then T is proved, relative to the imported lemmas.*

Proof. List the vertices of G in document order as $v_1, \dots, v_n$. By **(H6)**, v_i cites only v_j with $j < i$. If $v_1, \dots, v_{i-1}$ are established, so is v_i, as it was checked from exactly these. By induction all are established, and $v_n = T$ by **(H7)**.[2] □

Corollary 5.3 (Order independence)**.** *Vertices may be checked in any order, over any number of sittings, and by any number of people. The total effort can be estimated in advance: at most $|G|$ proofs, each shorter than L visible characters.*

In particular a reading group can share the work, each member taking a region of the map, and the reader never needs to hold more than one short proof, together with the statements it cites, in mind at once.

5.2 A checking procedure

Procedure 5.4 (Checking an HFF document)**.**

Step 1. Orient. Read Part I, open the map, and travel to T.

[2] In keeping with the subject, this proof is within budget in the sense of Definition 4.1: its length is 182.

Step 2. Check the target. Confirm that the statement of T in Part II is exactly the theorem you wanted proved, and that every definition it depends on means what you expect. A correct proof of a subtly different statement is the most insidious failure, and no amount of checking below T will detect it.

Step 3. Check the trust boundary. Compare every imported lemma with its source.

Step 4. Check the blocks. Check every remaining vertex, marking each as verified on the map. Working downward from T is usually best, since each lemma is then met already knowing what it is used for, but by Corollary 5.3 any order is valid.

Step 5. Finish. When the map has no fog left, T is proved relative to the imported lemmas, by Proposition 5.2.

5.3 When a block does not convince

A proof can be within budget and still fail to convince a particular reader, since short is not the same as obvious. The difficulty is confined to one vertex, so the remedy is local.

Definition 5.5 (Refinement)**.** A *refinement* of a lemma v is an HFF fragment whose main theorem is the statement of v, and whose new blocks may cite each other and the blocks that v cites. Replacing the proof of v by the refinement gives a new document that again satisfies **(H6)** and **(H7)**, whose graph is G with new vertices inserted below v.

In practice refinement is a prompt, "refine Lemma 12", and on the map the new vertices appear just below v. If v still resists checking after refinement, the problem has been localised to a single small claim, which is far more useful than the familiar report that a proof "is wrong somewhere on page 23". If the AI then repairs v and its statement changes, only the blocks that cite v need to be checked again, and the map shows exactly which they are.

6 Producing HFF documents

The following template, sent together with the statement of T (and, if one exists, an earlier proof to be rewritten), specifies the form.

Prompt template

Prove the following theorem and present the proof in *Human Friendly Form*, as specified below.

Theorem. ⟨statement⟩

Part I (overview). Begin with at most two pages of English prose, using as few formulas as possible. State the theorem in words, describe the overall strategy, name each method or technique used, say which steps are routine and which are delicate, and name the key lemmas by their numbers in Part II.

Separator. Then output a line containing only the symbol ✠.

Part II (proof).

1. If there are at most 15 definitions and lemmas, begin with a drawing of the dependency graph, with an arrow from u to v whenever v uses u and the main theorem at the top.
2. Apart from that drawing, all text in Part II must lie inside a Definition, Lemma or Remark environment. Each definition and lemma has a header of at most six words.

3. Each lemma's proof comes directly after its statement and, once the document is compiled, must contain fewer than 300 visible characters: every letter, digit, punctuation mark and mathematical symbol that appears on the page counts as one, whether it is in running text or in a formula, inline or displayed; spaces do not count. Any argument that does not fit must be split into further lemmas.
4. Each block may use only blocks that appear earlier, and must cite them explicitly by number. The main theorem is the last lemma, and every block must be used, directly or indirectly, by the main theorem.
5. Remarks may give motivation and warnings, but must not be needed for the proof, and no block may cite a remark.
6. Any existence, uniqueness or well-definedness claim needed by a definition is a separate lemma, placed before the definition.
7. Results from the literature are stated as Imported Lemmas, with a precise citation (including the theorem number) in place of a proof.

References. End the document with a list of references, and cite it wherever a result or idea comes from the literature.

The map. As a separate file, output the map data in JSON: one entry per definition and lemma (id, kind, number, header, statement, proof, length, and the ids of the blocks it cites), and the references. Remarks are not included in the map.

Conditions **(H2)**–**(H9)** are syntactic, and a short script can check them: it locates the separator, confirms that all text after it lies in allowed environments, compiles the document and measures the length of every proof (Definition 4.1), extracts the citations and confirms that they point only backwards with T as the unique sink, confirms that every citation of the literature resolves, and compares the map data with the document. The natural workflow is a loop: generate, check the format, return the violations to the AI, and repeat. Lamport's structured proofs [3, 4] are an antecedent with the same aim of making every step explicit; HFF adds a bound on the size of each step and organises the steps by a graph rather than a hierarchy.

7 A worked example

We close with a complete, if tiny, HFF document; the theorem is classical and the point is the form. The length of each proof (Definition 4.1) is shown in grey at the end of the proof; it was measured by counting the non-space characters in the text extracted from the compiled PDF. Since the graph is small, the optional static drawing is included. A prototype of the map for this example is in the file `hff_map_example.html`.

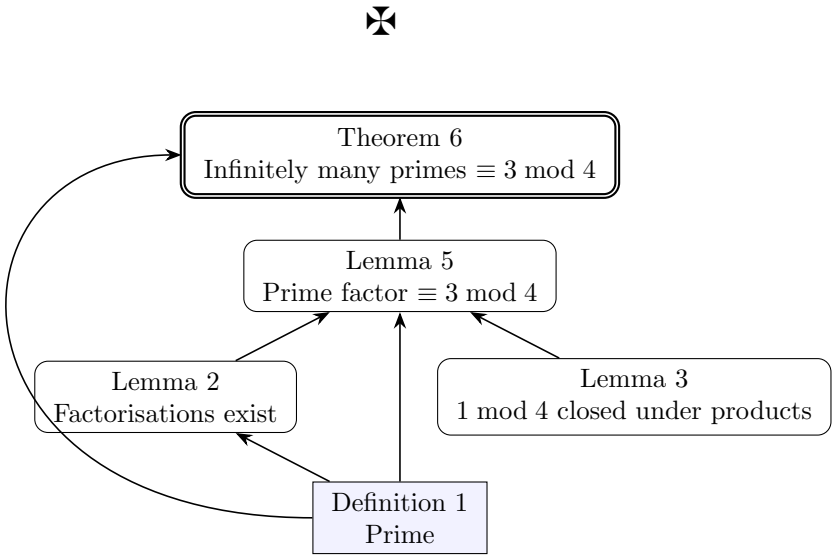
An HFF document

Overview. *Theorem.* There are infinitely many prime numbers that leave remainder 3 on division by 4.

Method. The proof is a variant of Euclid's argument for the infinitude of primes [Euc]. Suppose there were only finitely many such primes. Multiply them all together, multiply by 4, and subtract 1. The resulting number N leaves remainder 3 on division by 4. The key observation (Lemma 5) is that any number of this kind has a prime factor which also leaves remainder 3, because numbers leaving remainder 1 are closed under multiplication

(Lemma 3). But no prime on our list divides N, since each divides $N+1$, and this is a contradiction.

Structure. Lemmas 2 and 3 are routine, Lemma 5 is the delicate step, and Theorem 6 is Euclid's trick. Prerequisites: divisibility and congruences.



Definition 1 (Prime)**.** An integer $p > 1$ is *prime* if its only positive divisors are 1 and p.

Lemma 2 (Factorisations exist)**.** *Every integer $n > 1$ is a product of primes.*

Proof. We use strong induction on n. If n is prime, we are done. Otherwise, by Definition 1, $n = ab$ for some integers $1 < a, b < n$. By the induction hypothesis a and b are products of primes, and hence so is n. [length 159/300] □

Lemma 3 (1 mod 4 closed under products)**.** *If $a \equiv b \equiv 1 \pmod 4$, then $ab \equiv 1 \pmod 4$.*

Proof. Write $a = 4s + 1$ and $b = 4t + 1$ with s, t integers. Then

$$ab = 16st + 4s + 4t + 1 = 4(4st + s + t) + 1.$$ [length 69/300] □

Remark 4 (What comes next). Lemmas 2 and 3 are routine. The next lemma is the heart of the argument: it combines them to show that the residue 3 cannot be built out of primes that are all $\equiv 1 \pmod 4$.

Lemma 5 (Prime factor $\equiv 3 \bmod 4$)**.** *If $n > 1$ and $n \equiv 3 \pmod 4$, then some prime $p \equiv 3 \pmod 4$ divides n.*

Proof. By Lemma 2, $n = p_1 \cdots p_r$ with each p_i prime. Since n is odd, each p_i is odd, so $p_i \equiv 1$ or $3 \pmod 4$. If every $p_i \equiv 1 \pmod 4$, then $n \equiv 1 \pmod 4$ by Lemma 3 applied $r - 1$ times, which is a contradiction. [length 151/300] □

Theorem 6 (Infinitely many primes $\equiv 3 \bmod 4$)**.** *There are infinitely many primes $p \equiv 3 \pmod 4$.*

Proof. The prime 3 is one. Suppose $p_1, \dots, p_k$ are all of them, and let $N = 4p_1 \cdots p_k - 1$. Then $N > 1$ and $N \equiv 3 \pmod 4$, so by Lemma 5 some prime $p \equiv 3 \pmod 4$ divides N. Then $p = p_i$ for some i, so p divides $4p_1 \cdots p_k - N = 1$, which is absurd. [length 174/300] □

Remark 7 (Why subtract one). The -1 is what makes $N \equiv 3 \pmod 4$. Taking $N = 4p_1 \cdots p_k + 1$ instead gives $N \equiv 1 \pmod 4$, and Lemma 5 says nothing about such numbers.

Remark 8 (The residue 1). There are also infinitely many primes $\equiv 1 \pmod 4$, by Dirichlet's theorem on primes in arithmetic progressions [Dir], but this argument does not show it. The analogue of Lemma 3 for the residue 3 fails, since $3 \cdot 3 \equiv 1 \pmod 4$, and so a number $\equiv 1 \pmod 4$ need not have a prime factor $\equiv 1 \pmod 4$; for example, $21 = 3 \cdot 7$.

References

[Dir] P. G. L. Dirichlet, *Beweis des Satzes, dass jede unbegrenzte arithmetische Progression, deren erstes Glied und Differenz ganze Zahlen ohne gemeinschaftlichen Factor sind, unendlich viele Primzahlen enthält*, Abh. Königl. Preuss. Akad. Wiss. Berlin (1837), 45–81.

[Euc] Euclid, *Elements*, Book IX, Proposition 20.

A reader following Procedure 5.4 would read the overview and travel to Theorem 6 on the map; confirm that it states the intended theorem and that Definition 1 is the usual notion of prime; note that there are no imported lemmas, since the references are cited only in the overview and a remark; and then check Theorem 6, Lemma 5, Lemma 2 and Lemma 3 in that order, each a single short proof. With the definition already confirmed, the map is then clear of fog and the theorem is proved.

8 Open questions

(1) **Calibration.** What value of L makes checking fastest and most reliable, and how does it depend on the field?

(2) **Scale.** For a proof with thousands of lemmas, the map needs more structure, for instance modules that appear as named regions when zoomed out and open into their lemmas when zoomed in. Which layouts stay readable at that scale and stable under refinement?

(3) **Good decompositions.** **(H5)** can be met by chopping a proof into many lemmas with no natural content (Remark 4.4). Is there a useful measure of the quality of a decomposition?

(4) **Evidence.** Do readers in fact find errors faster and more reliably in HFF than in a conventional write-up? One test is to plant errors in AI-generated proofs and compare how often they are caught in each form.

Acknowledgements

The core idea of this note is the author's: asking an AI to present its proof in the Human Friendly Form, checking that proof one short lemma at a time, and finding one's way through it on an interactive map of its dependency graph. Claude, an AI model developed by Anthropic, was responsible for the write-up under the author's direction. It put the author's ideas into

clear English, typeset the note in LaTeX, drew the figures, and built the worked example and the prototype map. The rule in Section 4 for measuring the length of a proof, by counting the visible characters of the compiled document with no exemption for formulas, is the author's; it replaces a rule proposed by Claude in an earlier version of this note, which treated formulas separately. Some other details were proposed by Claude and adopted by the author, such as conditions **(H6)**–**(H8)**.

References

[1] J. Alper, M. Barany, A. Chavarri Villarello, S. Dahmen, W. Dean, K. Ganapathy, M. Harris, D. Holmes, M. Jamnik, S. Kelk, B. Kra, U. Martin, B. Naskręcki, R. Ochigame, J. Portegies and J. Schmitt, *Leiden Declaration on Artificial Intelligence and Mathematics*, Zenodo, 2 June 2026, doi:10.5281/zenodo.20302944. Available at `https://leidendeclaration.ai/`.

[2] Ensemble Studios, *Age of Empires*, video game, Microsoft, 1997.

[3] L. Lamport, *How to write a proof*, Amer. Math. Monthly **102** (1995), no. 7, 600–608.

[4] L. Lamport, *How to write a 21st century proof*, J. Fixed Point Theory Appl. **11** (2012), 43–63.

[5] T. Tao, *Mathematics in the age of AI*, preprint, 2026, arXiv:2608.16753 [math.HO].